This lecture explains the concept of "duck typing" in programming. Duck typing essentially means that the type or class of an object is less important than the methods it defines or implements. The lecture uses a Python example to illustrate this.

Key points:

- Definition of Duck Typing: Duck typing is a style of dynamic typing in which an object's suitability is determined by the presence of certain methods and properties, rather than its actual type or class. If it walks like a duck and quacks like a duck, then it is a duck.
- Example: A class `Course` is defined with an `__init__` method that takes `self` and `topics` as arguments, and a special method `__len__` that returns the length of `self.topics`.
- Demonstration: The lecture demonstrates that the `__len__` method works correctly whether `topics` is initialized with a string, a list, or a tuple. This is because the `__len__` method relies on the object passed to `topics` having a length that can be determined by the `len()` function.
- Irrespective of Data Type: Duck typing allows the `__len__` method to work with various data types as long as they support the `len()` function. The specific type of the data doesn't matter as long as it behaves as expected.
- Analogy: The `__len__` method is described as behaving like a "duck" because it works with any object that has a length, regardless of its specific type.
- Main Takeaway: Duck typing promotes flexibility and code reuse by focusing on behavior rather than strict type checking. It allows functions and methods to work with a wider range of objects, as long as those objects implement the necessary methods or properties.
-